

Day 4 — DSA: Sliding Window

Textbook-Style Detailed Notes

Sliding Window is one of the most important DSA patterns for Google interviews. It is particularly useful when a problem asks you to reason about a contiguous subarray or substring.

The key idea is:

Instead of recomputing every possible contiguous range from scratch, maintain a window and efficiently move its boundaries.

For Day 4, you should learn Sliding Window at three levels:

- Fixed-size window
- Variable-size window
- Advanced windows using frequency maps, sets, and other state

1. What Is a Sliding Window?

A window is a contiguous portion of an array or string.

For example:

```
nums = [2, 1, 5, 1, 3, 2]
```

A window might be:

```
[2, 1, 5]
```

or:

```
[5, 1, 3]
```

The window is defined by two boundaries:

```
left
↓
[2, 1, 5, 1, 3, 2]
      ↑
      right
```

The two pointers are usually:

```
left
right
```

But unlike the classic opposite-direction Two Pointer technique, both boundaries generally move from left to right.

2. Why Do We Need Sliding Window?

Suppose the problem asks:

Find the maximum sum of any subarray of size `k`.

Given:

```
[2, 1, 5, 1, 3, 2]
```

and:

```
k = 3
```

Possible windows:

[2, 1, 5] → 8

[1, 5, 1] → 7

[5, 1, 3] → 9

[1, 3, 2] → 6

Answer:

9

A naive approach calculates each sum independently.

For example:

```
2 + 1 + 5
1 + 5 + 1
5 + 1 + 3
1 + 3 + 2
```

There is massive overlap.

Compare:

[2, 1, 5]
 ↓ ↓

[1, 5, 1]

We don't need to recalculate `1 + 5`.

We can:

```
old window
[2, 1, 5]
  ↑   ↑
remove 2
  add 1

new window
[1, 5, 1]
```

This is the fundamental optimization.

3. The Core Sliding Window Idea

Instead of:

```
Calculate window 1 from scratch
Calculate window 2 from scratch
Calculate window 3 from scratch
...
```

we do:

```
Build first window
      ↓
Remove element leaving window
      ↓
Add element entering window
      ↓
Update answer
```

Conceptually:

```
[ A B C ] D E F
  ↓
[ A B C ] D E F
```

```
remove A
add D
```

```
A [ B C D ] E F
```

This turns many $O(n \times k)$ solutions into $O(n)$.

4. When Should You Think "Sliding Window"?

Look for these phrases:

- contiguous subarray
- contiguous substring
- consecutive elements
- consecutive characters
- subarray of size k
- substring of length k
- longest substring
- shortest subarray
- maximum/minimum sum
- at most k
- at least k
- no repeating characters
- contains all characters
- distinct elements
- frequency constraints

The strongest clue is:

The answer involves a contiguous range.

5. Sliding Window vs Subsequence

This distinction is extremely important.

Subarray

Must be contiguous.

```
[1, 2, 3, 4, 5]
```

```
[2, 3, 4]
```

Valid.

Subsequence

Doesn't have to be contiguous.

```
[1, 2, 3, 4, 5]
```

```
1, 3, 5
```

Valid subsequence.

```
1, 3, 5
```

is not a contiguous subarray.

Sliding Window is primarily designed for contiguous ranges.

6. Two Major Types

There are two major forms:

Fixed-size Sliding Window

The window size is known.

Find maximum sum of subarray of size k.

Window:

$\text{right} - \text{left} + 1 = k$

Variable-size Sliding Window

The window expands and contracts according to a condition.

Find the longest substring without repeating characters.

The size isn't predetermined.

1
2
3
2
4
...

7. Fixed-Size Sliding Window

Let's start with the simplest and most important version.

Problem

Given:

```
nums = [2, 1, 5, 1, 3, 2]
k = 3
```

Find the maximum sum of any subarray of size 3.

Brute Force

```
def max_sum_brute(nums, k):
    best = float("-inf")

    for i in range(len(nums) - k + 1):
        current = 0

        for j in range(i, i + k):
            current += nums[j]

        best = max(best, current)

    return best
```

Complexity:

Time: $O(nk)$
Space: $O(1)$

If:

$n = 1,000,000$
 $k = 500,000$

this becomes extremely expensive.

8. Sliding Window Optimization

Start with:

[2, 1, 5]

Sum:

8

Move one position:

[1, 5, 1]

Instead of recalculating:

1 + 5 + 1

do:

8 - 2 + 1 = 7

Move again:

[5, 1, 3]

7 - 1 + 3 = 9

Again:

[1, 3, 2]

9 - 5 + 2 = 6

Maximum:

9

9. Fixed Window Implementation

```
def max_sum_subarray(nums, k):  
    window_sum = sum(nums[:k])  
    best = window_sum  
  
    for right in range(k, len(nums)):  
        window_sum += nums[right]  
        window_sum -= nums[right - k]  
  
        best = max(best, window_sum)  
  
    return best
```

Complexity:

Time: $O(n)$
Space: $O(1)$

10. Why Is It $O(n)$?

The first window takes:

$O(k)$

Then every subsequent element is:

added once

and:

removed once

Therefore:

$$O(k) + O(n-k)$$

which simplifies to:

$$O(n)$$

This is an important interview explanation.

11. The Fixed Window Template

A useful template:

```
window_state = initial_window()

for right in range(k, len(nums)):
    add(nums[right])
    remove(nums[right - k])

    update_answer(window_state)
```

Or using explicit `left` and `right`:

```
left = 0

for right in range(len(nums)):
    add(nums[right])

    if right - left + 1 == k:
        update_answer()

        remove(nums[left])
        left += 1
```

The second version is useful because it makes the window boundaries explicit.

12. Fixed Window Example — Maximum Average

Problem

Given an integer array and integer `k`, find the maximum average of a contiguous subarray of size `k`.

Example:

```
nums = [1, 12, -5, -6, 50, 3]
k = 4
```

Windows:

```
[1, 12, -5, -6] → 2
[12, -5, -6, 50] → 51
[-5, -6, 50, 3] → 42
```

Maximum sum:

51

Maximum average:

$$51 / 4 = 12.75$$

```
def find_max_average(nums, k):
    window_sum = sum(nums[:k])
    best = window_sum

    for right in range(k, len(nums)):
        window_sum += nums[right]
        window_sum -= nums[right - k]

        best = max(best, window_sum)

    return best / k
```

13. Variable-Size Sliding Window

Now things become more interesting.

Suppose the problem says:

Find the longest subarray whose sum is at most `k`.

Example:

```
nums = [2, 1, 3, 2, 1]
k = 6
```

We don't know the window size beforehand.

It could be:

```
[2]
[2,1]
[2,1,3]
[2,1,3,2] ← sum too large
```

Therefore we need to dynamically expand and shrink the window.

14. The Basic Variable Window Structure

The general structure is:

```
left = 0

for right in range(len(nums)):

    # Add nums[right]

    while window_is_invalid:
        # Remove nums[left]
        left += 1

    # Window is now valid
    update_answer()
```

This is one of the most important templates in DSA.

15. Why Do We Use `while`, Not Just `if`?

Suppose:

```
window = [5, 2, 4]
sum = 11
k = 6
```

Removing one element:

```
remove 5
```

gives:

```
[2, 4]
sum = 6
```

Now valid.

But consider:

```
window = [8, 7, 6]
sum = 21
k = 6
```

Remove `8`:

```
[7, 6]
```

```
sum = 13
```

Still invalid.

Remove `7`:

```
[6]
sum = 6
```

Now valid.

Therefore:

```
while invalid:
```

is required when multiple elements may need to be removed.

16. Longest Subarray With Sum $\leq K$

Assume all numbers are non-negative.

```
def longest_subarray(nums, k):
    left = 0
    window_sum = 0
    best = 0

    for right in range(len(nums)):
        window_sum += nums[right]

        while window_sum > k:
            window_sum -= nums[left]
            left += 1

        best = max(best, right - left + 1)

    return best
```

17. Dry Run

Given:

```
nums = [2, 1, 3, 2, 1]
k = 6
```

Start:

```
left = 0
```

Add 2

```
[2]
sum = 2
```

Length:

```
1
```

Best:

```
1
```

Add 1

```
[2,1]
sum = 3
```

Length:

```
2
```

Best:

2

Add 3

```
[2,1,3]
sum = 6
```

Valid.

Length:

3

Best:

3

Add 2

```
[2,1,3,2]
sum = 8
```

Invalid.

Shrink:

```
remove 2
```

```
[1,3,2]
sum = 6
```

Length:

3

Best remains:

3

Add 1

```
[1,3,2,1]
sum = 7
```

Invalid.

Remove `1`:

```
[3,2,1]
sum = 6
```

Length:

3

Final answer:

3

18. The Most Important Variable Window Invariant

For the previous problem:

After the shrinking loop finishes, the current window always satisfies `window_sum <= k`.

That is the invariant.

The algorithm maintains:

```
expand
↓
possibly invalid
↓
```

```
shrink until valid
↓
record answer
```

This structure is extremely powerful.

19. Why Does Sliding Window Work?

Sliding Window relies on a form of monotonicity.

Suppose all numbers are non-negative.

If:

```
window_sum > k
```

then removing elements from the left cannot increase the sum.

Therefore:

```
left++
```

moves us toward a valid window.

This is why the technique works.

20. Important Limitation: Negative Numbers

This is a critical interview concept.

Consider:

```
nums = [5, -10, 5]
k = 5
```

With negative values, expanding the window can actually decrease the sum.

Therefore the simple rule:

```
if sum > k:
    shrink
```

may not give the correct answer for every problem.

This means:

You cannot blindly apply Sliding Window to every subarray-sum problem.

When negative numbers are involved, techniques such as:

- prefix sums
- hash maps
- monotonic queues
- binary search on prefix sums

may be required depending on the problem.

This distinction is extremely valuable in interviews.

21. Longest Substring Without Repeating Characters

This is one of the most famous Sliding Window problems.

Problem

Given:

"abcbcbcb"

find the length of the longest substring without repeated characters.

Answer:

3

because:

"abc"

22. Why Sliding Window?

We need a:

contiguous substring

and we need to maintain the property:

no duplicate characters

Therefore maintain:

```
left
right
set of characters in current window
```

23. Algorithm

Start:

"a"

Valid.

Expand:

"ab"

Valid.

Expand:

"abc"

Valid.

Next:

"abca"

Invalid because 'a' repeats.

We shrink from the left until the window becomes valid again.

"bca"

Now valid.

24. Implementation

```
def length_of_longest_substring(s):
    left = 0
    seen = set()
    best = 0

    for right in range(len(s)):
        while s[right] in seen:
            seen.remove(s[left])
            left += 1
```

```

        seen.add(s[right])

        best = max(best, right - left + 1)

    return best

```

Complexity:

```

Time:  O(n)
Space: O(min(n, alphabet_size))

```

25. Why Is This $O(n)$ Despite the Nested While Loop?

This is a classic interview question.

Someone might see:

```

for right in range(n):
    while ...:
        left += 1

```

and think:

$O(n^2)$

But that's not necessarily true.

`right` moves:

$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow n-1$

and `left` also moves:

$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow n-1$

Neither pointer moves backward.

Therefore:

```

right moves at most n times
left moves at most n times

```

Total:

$O(2n)$

which is:

$O(n)$

This is called amortized analysis.

26. Amortized Analysis

This is a concept worth understanding for Google interviews.

Suppose:

```

for right:
    while condition:
        left += 1

```

The `while` loop can execute many times for one particular `right`.

But across the entire algorithm:

left

can only move forward `n` times.

Therefore total pointer movement is bounded by:

$n+n=2n$

Thus:

$O(n)$

The important idea:

Analyze the total number of pointer movements, not the apparent nesting of loops.

27. Sliding Window With Frequency Maps

Sometimes we need more information than a set.

Suppose the problem asks:

Find the longest substring containing at most two distinct characters.

Example:

"eceba"

Possible answer:

"ece"

Length:

3

We need to know:

how many times each character occurs

So use a dictionary:

count = {}

28. At Most K Distinct Characters

```
def longest_substring_k_distinct(s, k):
    left = 0
    count = {}
    best = 0

    for right in range(len(s)):
        char = s[right]
        count[char] = count.get(char, 0) + 1

        while len(count) > k:
            left_char = s[left]
            count[left_char] -= 1
            if count[left_char] == 0:
                del count[left_char]

            left += 1

        best = max(best, right - left + 1)

    return best
```

The invariant:

After the while loop, the window contains at most `k` distinct characters.

29. Set vs Frequency Map

Use a set when you only care whether something exists.

Example:

Does this character already exist?

Use a frequency map when you care about counts.

Example:

```
How many times does this character occur?
```

This distinction is important.

30. Minimum Window Problems

Another major Sliding Window category is:

Find the smallest window satisfying a requirement.

Example:

```
s = "ADOBECODEBANC"
t = "ABC"
```

Answer:

```
"BANC"
```

The strategy is different from a simple longest-window problem.

We usually:

```
expand right
  ↓
eventually satisfy requirement
  ↓
shrink left as much as possible
  ↓
record minimum
```

This is a key pattern.

31. Maximum vs Minimum Windows

Longest valid window

Usually:

```
expand
shrink while invalid
update maximum
```

Example:

```
longest substring with ≤ k distinct
```

Smallest valid window

Usually:

```
expand
while valid:
  update minimum
  shrink
```

Example:

```
minimum window substring
```

This difference is extremely useful.

32. General Sliding Window Templates

Template A — Fixed Size

```
left = 0

for right in range(len(nums)):

    add(nums[right])

    if right - left + 1 == k:
        update_answer()

    remove(nums[left])
    left += 1
```

Template B — Longest Valid Window

```
left = 0

for right in range(len(nums)):

    add(nums[right])

    while window_is_invalid:
        remove(nums[left])
        left += 1

    update_maximum()
```

Template C — Smallest Valid Window

```
left = 0

for right in range(len(nums)):

    add(nums[right])

    while window_is_valid:
        update_minimum()

    remove(nums[left])
    left += 1
```

These three templates cover a huge number of interview problems.

33. Window State

A very important concept is window state.

The window itself is:

```
[left ... right]
```

But we often maintain additional information about it.

Examples:

Sum

```
window_sum
```

Frequency

```
count = {}
```

Distinct values

```
set()
```

Number of zeros

```
zero_count
```

Number of matching characters

matched

Maximum/minimum

Sometimes:

deque

The state lets us evaluate the current window efficiently.

34. Don't Recalculate the Window

Bad:

```
current_sum = sum(nums[left:right + 1])
```

inside every iteration.

That can lead to:

$O(n^2)$

Instead maintain:

```
entering element → add  
leaving element → remove
```

This is the central optimization.

35. Sliding Window With Counts

Consider:

Find the longest substring containing no more than two occurrences of any character.

Example:

"ccaabbb"

You might maintain:

```
count = {}
```

When adding:

```
count[s[right]] += 1
```

When shrinking:

```
count[s[left]] -= 1
```

When a count reaches zero:

```
del count[s[left]]
```

The key is that the window state changes incrementally.

36. Another Classic: Fruit Into Baskets

This is a famous variable-window problem.

The problem can be interpreted as:

Find the longest contiguous subarray containing at most two distinct values.

Example:

```
[1, 2, 1, 2, 3]
```

The longest valid window is:

```
[1, 2, 1, 2]
```

Length:

```
4
```

This is essentially:

Longest subarray with at most 2 distinct values

```
def total_fruit(fruits):
    left = 0
    count = {}
    best = 0

    for right in range(len(fruits)):
        count[fruits[right]] = count.get(fruits[right], 0) + 1

        while len(count) > 2:
            value = fruits[left]
            count[value] -= 1

            if count[value] == 0:
                del count[value]

            left += 1

        best = max(best, right - left + 1)

    return best
```

37. Another Classic: Longest Repeating Character Replacement

Problem:

```
s = "AABABBA"
k = 1
```

Find the longest substring that can be transformed into a string of the same character by replacing at most `k` characters.

The key window condition is:

```
window_length - most_frequent_character_count <= k
```

Why?

Suppose:

```
window = "AABAB"
```

Length:

```
5
```

Most frequent:

```
A → 3
```

Characters needing replacement:

```
5 - 3 = 2
```

If:

```
k = 1
```

window is invalid.

Shrink.

38. Implementation

```
def character_replacement(s, k):
    left = 0
    count = {}
    max_frequency = 0
    best = 0

    for right in range(len(s)):
        char = s[right]

        count[char] = count.get(char, 0) + 1

        max_frequency = max(
            max_frequency,
            count[char]
        )

        while (right - left + 1) - max_frequency > k:
            left_char = s[left]
            count[left_char] -= 1
            left += 1

        best = max(
            best,
            right - left + 1
        )

    return best
```

This is an excellent example of maintaining a derived window state.

39. A Subtle Interview Question

Why don't we always decrease `max\_frequency` when removing a character?

Because we don't necessarily need the exact current maximum frequency.

We need a value that lets us determine whether the window can potentially produce a better answer.

Keeping the historical maximum can still produce a correct result because window lengths and the expansion process constrain what can be recorded.

This is an example of an important interview lesson:

The state you maintain does not always need to be recomputed exactly if you can prove that the maintained approximation cannot cause an incorrect answer.

40. Sliding Window + Deque

Some advanced problems ask:

Find the maximum value inside every window of size `k`.

Example:

```
nums = [1, 3, -1, -3, 5, 3, 6, 7]
k = 3
```

Output:

```
[3, 3, 5, 5, 6, 7]
```

A naive solution scans every window:

```
O(nk)
```

A better solution uses a monotonic deque:

```
O(n)
```

This is still a sliding-window problem, but requires another data-structure pattern.

You should learn this after becoming comfortable with basic windows.

41. Sliding Window + Prefix Sum

Sliding Window and Prefix Sum can solve overlapping classes of problems, but they rely on different properties.

Sliding Window

Usually exploits:

```
monotonicity
```

For example:

```
adding elements makes condition worse  
removing elements makes condition better
```

Prefix Sum

Precomputes cumulative information:

```
prefix[i] = sum(nums[:i])
```

Then:

```
sum(left, right)
```

can be calculated efficiently.

Prefix sums become particularly important when negative numbers destroy the monotonicity needed by a simple sliding window.

42. How to Recognize Fixed vs Variable Window

Ask:

Question 1

Does the problem explicitly say:

```
subarray of size k  
substring of length k
```

Then:

Fixed window

Question 2

Does the problem say:

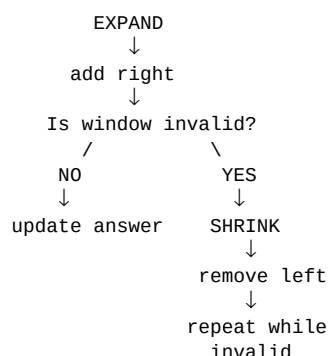
```
longest  
shortest  
at most  
at least  
no more than  
contains  
without repeating
```

Then investigate:

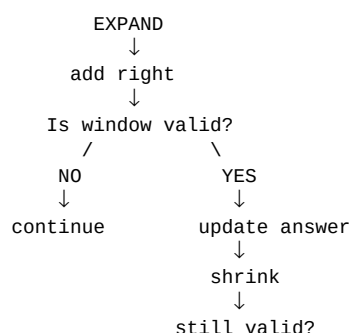
Variable window

43. The Most Important Pattern

For a variable window, remember:



For minimum-window problems:



44. The Most Important Invariant

For every Sliding Window problem, explicitly state:

What property is guaranteed to be true after I finish shrinking?

Examples:

At most K distinct

window contains $\leq K$ distinct elements

No duplicates

every character in window occurs once

Sum constraint

window_sum $\leq K$

Minimum window

window satisfies all required conditions

If you cannot state the invariant, you probably don't fully understand the algorithm yet.

45. Common Mistake #1 — Using Sliding Window for Non-Contiguous Problems

If the problem asks for:

subsequence

don't automatically use Sliding Window.

Sliding Window normally depends on:

contiguity

46. Common Mistake #2 — Forgetting Window Length

The window length is:

$\text{right} - \text{left} + 1$

Not:

$\text{right} - \text{left}$

Example:

$\text{left} = 2$
 $\text{right} = 5$

Elements are:

2, 3, 4, 5

Count:

4

Therefore:

$5 - 2 + 1 = 4$

47. Common Mistake #3 — Updating the Answer at the Wrong Time

For longest valid window:

```
while invalid:
    shrink()

best = max(best, window_length)
```

Do not update before restoring validity.

For minimum valid window:

```
while valid:
    best = min(best, window_length)
    shrink()
```

The timing matters.

48. Common Mistake #4 — Using `if` Instead of `while`

Incorrect in many variable-window problems:

```
if invalid:
    left += 1
```

Often you need:

```
while invalid:
    left += 1
```

because one removal may not restore validity.

49. Common Mistake #5 — Recomputing Window State

Avoid:

```
sum(nums[left:right + 1])
```

or repeatedly rebuilding a frequency map.

Instead:

```
entering element → add  
leaving element → remove
```

This is what gives Sliding Window its efficiency.

50. Common Mistake #6 — Ignoring Negative Numbers

Remember:

Simple sum-based Sliding Window often requires non-negative numbers.

If negative values exist, stop and ask:

Does the monotonicity assumption still hold?

If not, consider another technique.

51. Common Mistake #7 — Confusing Two Pointers and Sliding Window

They overlap, but their reasoning differs.

Two Pointers

Often:

```
left ↔ right
```

with movement based on comparisons.

Example:

- Two Sum
- Container With Most Water
- Palindrome

Sliding Window

Maintains:

```
[left ... right]
```

as a contiguous range with some state/property.

Example:

- Longest substring without repeats
- Maximum sum of size k
- Minimum window substring

A sliding window is often implemented using two pointers, but not every two-pointer algorithm is a sliding window.

52. Sliding Window vs Brute Force

Suppose:

```
n = 1,000,000
k = 100
```

Brute force:

```
approximately n × k
```

operations:

```
100 million
```

Sliding Window:

```
approximately n
```

operations:

```
1 million
```

This is why recognizing the pattern matters.

53. Interview Explanation Template

When presenting a Sliding Window solution, use this structure.

Step 1 — Identify contiguity

"The problem asks about a contiguous subarray/substring, so a sliding window is a candidate."

Step 2 — Define the state

"I'll maintain the current window and the information needed to determine whether it is valid."

Step 3 — Expand

"I move `right` to include the next element."

Step 4 — Restore validity

"If the window violates the constraint, I move `left` forward and remove elements until the window becomes valid."

Step 5 — Update answer

"Once the window is valid, I update the maximum/minimum length."

Step 6 — Complexity

"Both pointers move only forward, so the total number of pointer movements is $O(n)$."

This is a strong Google-style explanation.

54. The Deeper Concept: Maintain a Valid Region

Sliding Window is fundamentally about maintaining:

```
a dynamically changing valid region
```

Think:

```
[left ----- right]
      VALID
```

When adding an element makes it invalid:

```
[left ----- right]
      INVALID
```

we move:

```
left →
```

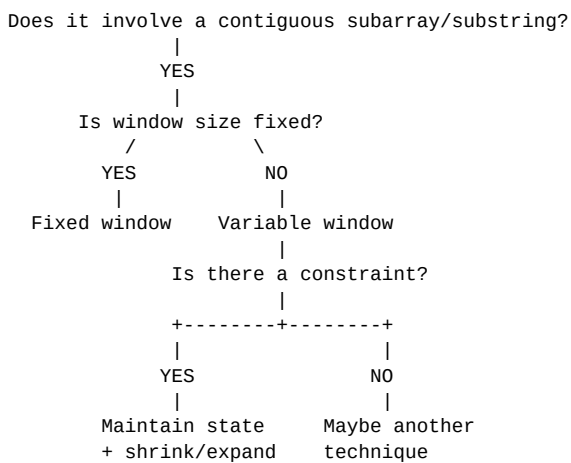
until:

```
[left ----- right]
      VALID
```

Then we continue expanding.

This mental model is more useful than memorizing code.

55. Sliding Window Decision Tree



Then ask:

```
What makes my window invalid?
```

and:

```
What operation restores validity?
```

56. Practice Progression

For Day 4, solve these in order.

Level 1 — Fixed Window

- 1. Maximum sum subarray of size `k`
- 2. Maximum average subarray of size `k`
- 3. Number of occurrences of anagram/permutation in fixed windows

Level 2 — Basic Variable Window

- 4. Longest subarray with sum $\leq k$ — non-negative numbers
- 5. Longest substring without repeating characters
- 6. Longest substring with at most `k` distinct characters

Level 3 — Advanced Variable Window

- 7. Fruit Into Baskets

- 8. Longest Repeating Character Replacement
- 9. Minimum Window Substring
- 10. Permutation in String

Level 4 — Advanced Data Structure

- 11. Sliding Window Maximum
- 12. Minimum Sliding Window
- 13. Subarrays with K Distinct Integers

57. Day 4 Sliding Window Definition of Done

Do not consider Sliding Window complete merely because you can reproduce a template.

You should be able to:

- Explain what a sliding window is.
- Explain why contiguity matters.
- Distinguish subarray from subsequence.
- Explain fixed-size windows.
- Explain variable-size windows.
- Implement a fixed-size window from memory.
- Implement a variable-size window from memory.
- Explain why $\text{right} - \text{left} + 1$ is the window size.
- Explain why `while` is often required.
- Explain amortized $O(n)$ complexity.
- Maintain a running sum.
- Maintain a set.
- Maintain a frequency dictionary.
- Identify the window invariant.
- Explain when to expand.
- Explain when to shrink.
- Know when to update the answer.
- Understand longest-valid-window problems.
- Understand minimum-valid-window problems.
- Explain why negative numbers can break simple sum-based windows.
- Distinguish Sliding Window from Two Pointers.
- Distinguish Sliding Window from Prefix Sum.
- Solve a new Sliding Window problem without memorizing its exact solution.

58. The One Sentence to Remember

Sliding Window maintains a contiguous region and incrementally adds elements on one side and removes elements from the other so that the required condition can be maintained efficiently.

And for Google interviews, remember the deeper principle:

Don't memorize "expand and shrink." Identify what makes the window valid, what makes it invalid, and why moving the left boundary is guaranteed to restore the invariant.

That reasoning is what turns Sliding Window from a memorized DSA trick into a pattern you can apply to unfamiliar Google interview problems.